Soumiya Ayyappan_W9_D3

Easy qn

Step 1:







Step 2:

```
from pyspark.sql.functions import when
df=df.dropna(subset=["amount"])
df=df.withColumn("amount_band",when(col("amount")>10000,"HIGH").otherwise("STANDARD"))
df=df.withColumn("review_flag",when(col("amount")>=20000,"REVIEW").otherwise("NORMAL"))
display(df.count())
display(df)
```

12:13 PM (6s) 4 Python

See performance (2) Checked for improvements

df: pyspark.sql.connect.dataframe.DataFrame = [txn_id: string, account_id: string ... 7 more fields]

9

Table +

	txn_id	account_id	channel	txn_type	amount	txn_status	txn_date	amount_band	review_fl
1	T001	A101	UPI	DEBIT	1200	SUCCESS	2026-09-01	STANDARD	NORMAL
2	T002	A102	ATM	DEBIT	5000	SUCCESS	2026-09-01	STANDARD	NORMAL
3	T003	A103	NEFT	CREDIT	18000	SUCCESS	2026-09-02	HIGH	NORMAL
4	T005	A105	IMPS	CREDIT	7500	SUCCESS	2026-09-03	STANDARD	NORMAL
5	T006	A106	ATM	DEBIT	3000	FAILED	2026-09-03	STANDARD	NORMAL
6	T007	A107	UPI	DEBIT	22000	SUCCESS	2026-09-04	HIGH	REVIEW
7	T008	A108	NEFT	CREDIT	14000	SUCCESS	2026-09-04	HIGH	NORMAL
8	T009	A109	IMPS	DEBIT	950	SUCCESS	2026-09-05	STANDARD	NORMAL
9	T010	A110	UPI	CREDIT	30000	SUCCESS	2026-09-05	HIGH	REVIEW

9 rows | 5.722s runtime Refreshed at 10/14/26, 12:13 PM (19m ago)

Step 3:

```
df.write.format("delta").mode("overwrite").saveAsTable("uc_practice_bank.transaction_ops.transaction_silver")
```

12:16 PM (8s) 5

See performance (1) Checked for improvements

```
display(spark.sql("select count(*) as total_count from uc_practice_bank.transaction_ops.transaction_silver"))
display(spark.sql("select count(*) as channel_count,sum(amount) as total_amount from uc_practice_bank.transaction_ops.transaction_silver group by channel order by channel;"))
```

12:21 PM (4s) 6

See performance (2) Checked for improvements

Table +

	total_count
1	9

1 row | 4.328s runtime

Table +

	channel_count	total_amount
1	2	8000
2	2	8450
3	2	32000
4	3	53200

Step 4:

12:29 PM (2s)

```
1 from pyspark.sql.functions import avg,round
2 display(spark.sql("select * from uc_practice_bank.transaction_ops.transaction_silver where review_flag in ('REVIEW','HIGH')"))
3 amt_avg=df.groupBy("channel").agg(round(avg(col("amount")),2).alias("avg_amount"))
4 amt_avg.show()
```

See performance (2) — Checked for improvements

amt_avg: pyspark.sql.connect.dataframe.DataFrame = [channel: string, avg_amount: double]

Table

	txn_id	account_id	channel	txn_type	amount	txn_status	txn_date	amount_band	review_flag
1	T007	A107	UPI	DEBIT	22000	SUCCESS	2026-09-04	HIGH	REVIEW
2	T010	A110	UPI	CREDIT	30000	SUCCESS	2026-09-05	HIGH	REVIEW

2 rows | 1.934s runtime — Refreshed at 10/14/26, 12:29 PM (8m ago)

```
+-------+----------+
|channel|avg_amount|
+-------+----------+
|   NEFT|   16000.0|
|   IMPS|    4225.0|
|    ATM|    4000.0|
|    UPI|  17733.33|
+-------+----------+
```